



University of Mindanao

College of Computing Education



Course: Software Development Fundamental

yUM-e: Food Ordering System

Submitted by:

<u>Abas, Emman</u>	<u>546488</u>
<u>Asan, Nino</u>	<u>546681</u>
<u>Bravo, Joana L.</u>	<u>546336</u>
<u>Nanding, Nazlah</u>	<u>545627</u>
<u>Hingpit, Hazeljoy T.</u>	<u>545666</u>

Instructor:
Modesto C. Tarrazona

Submission Date:
December 18, 2024

Table of Contents

Page

Introduction.....	3
Objectives.....	4
System Architecture	4
Class Diagram.....	10
Implementation Details.....	12
Key Features.....	18
Testing and Results.....	19
Conclusions.....	29
Future Enhancements.....	30
Source Code.....	32
Members Information.....	35

Introduction

This project aims to develop a solution that addresses the problem of long queues and delays at the university cafeteria during peak hours. By implementing a more efficient ordering and payment system, this project aims to reduce wait times, simplify the ordering process, and improve overall student satisfaction. This project's scope includes designing and implementing a digital platform that enables students to pre-order their meals, pay online, and select pick-up times to avoid congestion and waiting in line.

The motivation behind this project stems from the common frustrations students face during lunch and snack times. Long wait times not only waste valuable time but also contribute to stress and disrupt students' schedules. Given the increasing demand for food services on campus, coupled with limited staff and space, there is a need for a scalable solution that improves the cafeteria's service efficiency. By reducing queues and enhancing convenience, the system will allow students to spend less time waiting and more time enjoying their meals or engaging in other activities. The importance of this project lies in its potential to improve daily campus life, make better use of the cafeteria's resources, and create a more pleasant experience for the students and staff alike.

At our university cafeteria, students frequently experience long queues during lunch and snack times, resulting in significant delays when placing orders and paying for food. This inefficiency leads to frustration, wasted time, and students having less time to relax, eat, or make it to their next class. The problem is especially pronounced during peak hours when the staff struggles to process all orders promptly. This situation not only impacts students' satisfaction but also limits the overall capacity of the cafeteria to serve a larger volume of students within the available time frame. Thus, addressing these delays is crucial to enhancing both the operational efficiency of the cafeteria and the student experience.

Objectives

This project aims to reduce waiting times for students by allowing them to place meal orders in advance, making it quicker for them to pick up their food. It also focuses on improving the efficiency of canteen cashiers and staff by simplifying the ordering process, which will speed up service. The overall goal is to enhance the dining experience for both students and staff by simplifying order management and making the entire process more efficient, ultimately leading to higher satisfaction on all sides.

System Architecture

High-Level Architecture

Structure of the System

1. Modules:

- **Admin Module (YUM_e_AdminModule):**
 - Provides functionalities for admins to log in, view orders, update order statuses, and stocks.
 - Interacts with the database to fetch and modify order data.
- **LogIn Module (YUM_e_LogInModule):**
 - Handles student login using their username and ID.
 - Validates credentials against a database of verified students.
 - Redirects students to the menu module after successful login.
- **Menu Module (YUM_e_Menu):**
 - Allows students to select items (dishes, drinks, desserts) and specify quantities.
 - Displays a dynamic receipt, calculates the total bill, and manages order submission to the database.
 - Supports order types (Dine In or Take Out) and tracks order IDs.

2. **Database:**

- Contains tables like `verified_student` (for student credentials), `admin` (admin credentials), and `items` (order information, including items, order type, total bill, and timestamps).

3. **Interactions:**

- The **Login Module** validates credentials and transitions users (students or admins) to their respective modules.
- The **Admin Module** communicates with the database for administrative tasks.
- The **Menu Module** allows students to place orders, which are stored in the database.

Component Interaction

1. **User Actions:**

- Students and admins interact with the system through a **GUI** designed using Swing.
- Students log in via `YUM_e_LogInModule` and are directed to the `YUM_e_Menu` to place orders.
- Admins use `YUM_e_AdminModule` for managing orders.

2. **Database Operations:**

- All modules connect to the `admin_studentorders` database using JDBC.
- Queries are executed to validate credentials, fetch data, and update or insert new records.

3. **Order Workflow:**

- Students select items in the `YUM_e_Menu`, which dynamically updates the receipt and total.
- Once confirmed, the order is stored in the database.
- Admins can view these orders in the `YUM_e_AdminModule` and update their status.

Interaction Flow

1. When you first open the program, you will be greeted with a simple login screen featuring two text fields: “Name” and “Student ID.” In the first text field, you will enter your name, and in the second, you will input your Student ID. Please ensure that your inputs are correct to avoid errors when logging in. Once you have entered your details, click **Log-In** to proceed to the next screen.



Figure 1. Login Module

2. After clicking **Log-In**, you will be directed to a menu displaying a list of food items organized into categories. The categories include **Main Dishes**, **Drinks**, and **Desserts**.

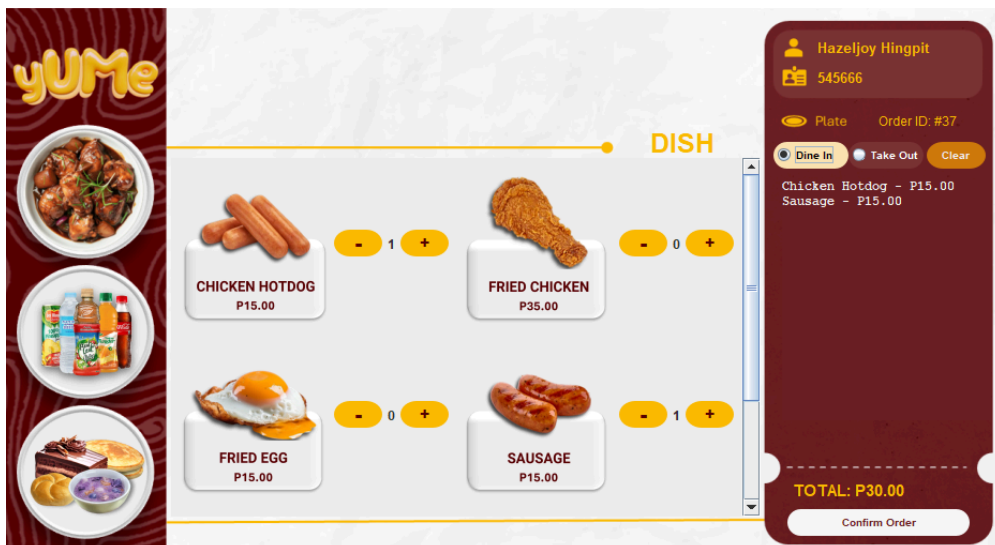


Figure 2. Menu

- When you confirm your order, an option pane will appear to validate your selection. After confirmation, you will receive your receipt, and the program will return to the login screen.

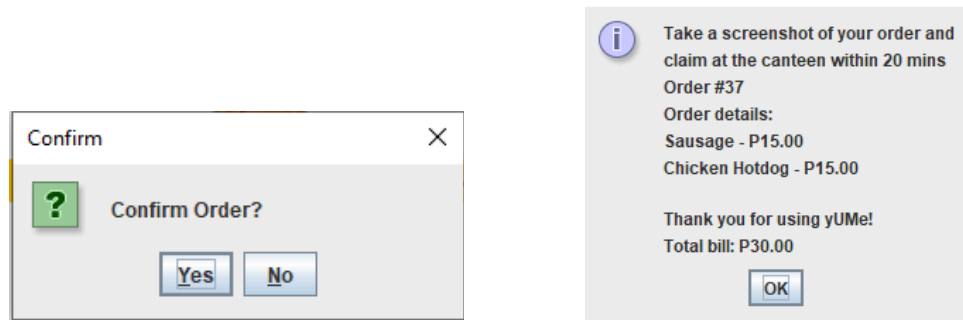


Figure 3. Confirmation

- In the **Admin Frame**, the administrator must log in.

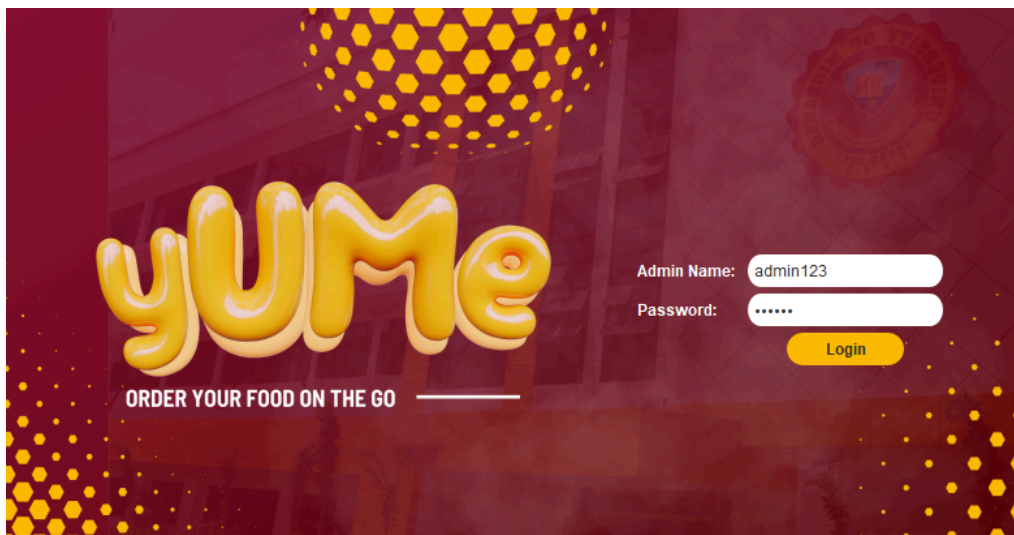


Figure 4. Admin Login Module

- After logging in, the administrator (cashier) will have access to the complete order history.

From here, the admin can:

- **Update the stock** of specific items when new inventory arrives.
- **Update the status** of an item to indicate that it is ready for pickup.

Update Stocks				
Order ID	Student ID	Item	Status	Order Time
29	2	Chicken Hotdog - P45.00	pending	2024-12-12 00:00:00.0
30	1	Chicken Hotdog - P45.00	pending	2024-12-12 00:00:00.0
31	1	Chocolate Cake - P1500.00	pending	2024-12-12 00:00:00.0
32	1	Bread - P60.00	pending	2024-12-12 00:00:00.0
33	2	Chicken Hotdog - P15.00	pending	2024-12-12 00:00:00.0
34	1	Fried Chicken - P35.00	pending	2024-12-12 00:00:00.0
35	2	Chicken Hotdog - P30.00	pending	2024-12-12 00:00:00.0
36	1	Fried Chicken - P140.00	ready	2024-12-12 00:00:00.0

Figure 5. Orders

- When you click **Update Status**, an option pane will pop up where you can enter the updated status of the item.

Success

Order status updated successfully.

Figure 6. Update Status

- To update stock levels, click the **Update Stocks** button. This will display the stock quantity for each item, and you can input a new stock value for a specific item.

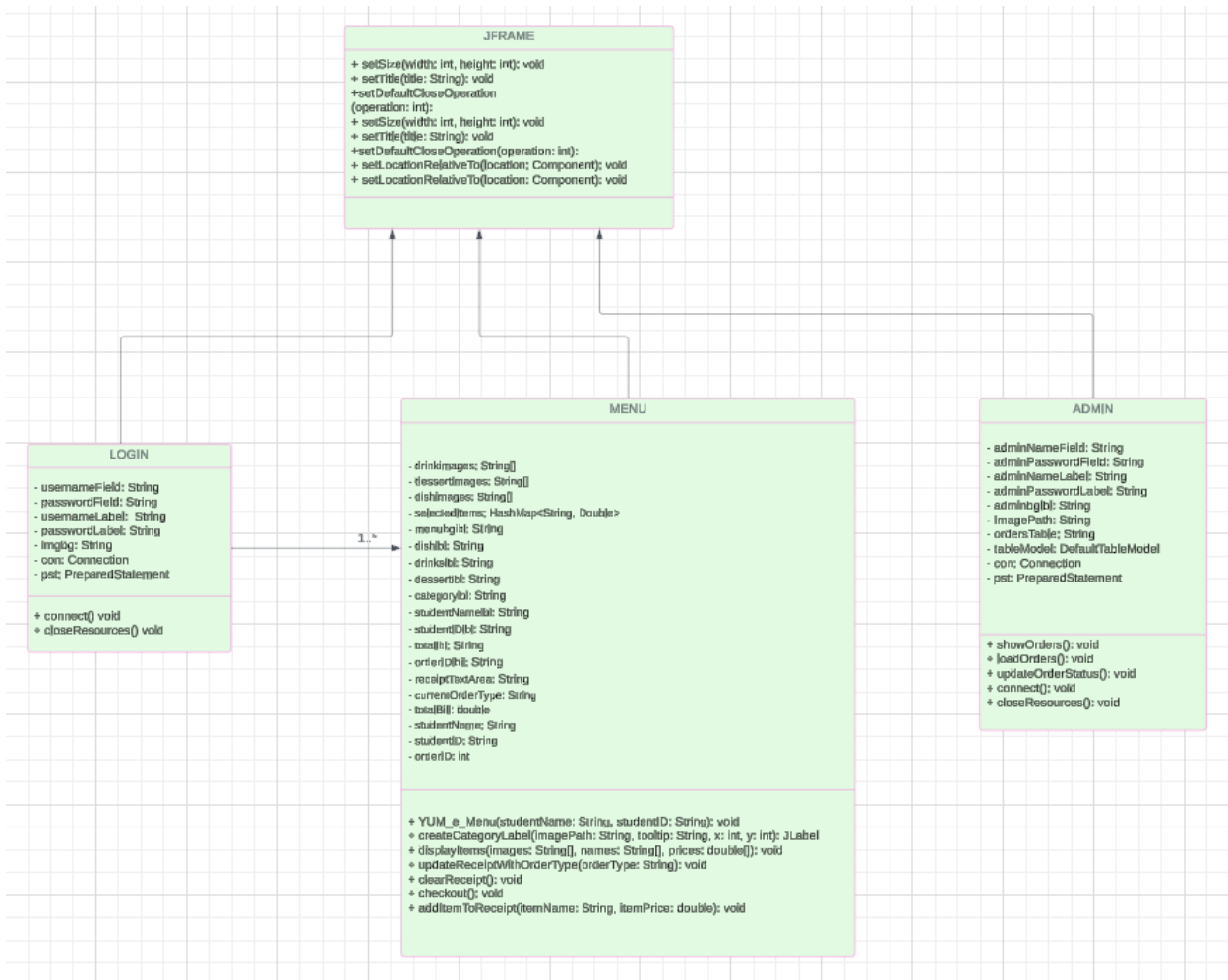
Food ID	Food Name	Price	Stock
1	Chicken Hotdog	15.0	50
2	Fried Chicken	35.0	50
3	Fried Egg	15.0	50
4	Sausage	15.0	50
5	Boiled Egg	15.0	50
6	Ginisang Toge	25.0	50
7	Real Leaf Apple	30.0	50
8	Tropicana Twister	30.0	50
9	Coke Mismo	20.0	50
10	Natures Spring	20.0	50
11	Del Monte	40.0	50
12	Gatorade Blue	40.0	50
13	Chocolate Cake	30.0	50
14	Ube Binignit	15.0	50
15	Bread	15.0	46
16	Pancake	15.0	50
17	Biko	15.0	50
18	Kutsinta	15.0	50
19	Chicken Hotdog	15.0	50

Figure 7. Update Stocks

Simplified interaction flow of the program:



Class Diagram



1. Inheritance Relationship (Generalization)

LOGIN, MENU, and ADMIN inherit from JFRAME.

This is shown by the solid arrows with hollow triangles pointing from the child classes (LOGIN, MENU, and ADMIN) to the parent class (JFRAME).

Meaning:

JFRAME acts as a **base class** that provides general functionalities for creating and managing GUI windows.

LOGIN, MENU, and ADMIN extend these capabilities to include their specific behavior.

All three classes leverage JFRAME's methods like setSize(), setTitle(), and setLocationRelativeTo() to control their respective GUI windows.

2. Association Relationship

Between LOGIN and MENU:

There is a 1.. (one-to-many)* association relationship between LOGIN and MENU.

This indicates that:

Once a user logs in through the LOGIN class, they can interact with or access multiple menus represented by the MENU class.

The association suggests that the LOGIN class likely initializes or calls methods from the MENU class after successful user authentication.

3. Composition/Dependency

Database Dependencies in LOGIN and ADMIN:

Both the LOGIN and ADMIN classes contain attributes like con: Connection and pst: PreparedStatement.

This shows that they depend on a database connection to perform tasks:

LOGIN uses the connection for user authentication.

ADMIN uses the connection for administrative tasks such as loading and updating orders.

Timer in ADMIN:

The ADMIN class includes a timer: Timer object.

This dependency indicates that the ADMIN class uses the Timer object to perform actions like canceling expired orders or automating certain tasks at scheduled intervals.

4. Aggregation of GUI Components

The attributes of LOGIN, MENU, and ADMIN classes (like usernameField, menuBgLbl, and ordersTable) suggest that these classes aggregate GUI components such as String fields, labels, and tables to construct user interfaces.

For Example:

LOGIN contains attributes like usernameField and passwordField to display text fields for user input.

MENU contains arrays (drinkImages[], dessertImages[]) and labels to display different food categories.

ADMIN contains ordersTable and tableModel to display order information in a tabular format.

Implementation Details

Key Classes and Objects

Class 1: YUM_e_AdminModule

Purpose:

The YUM_e_AdminModule class provides the GUI for the admin login and order management in the food ordering system. It allows the admin to log in, view orders, update order statuses, and interact with the database to manage orders.

Attributes:

- adminNameField: A text field for entering the admin's name.
- adminPasswordField: A password field for entering the admin's password.
- adminbg: An image object used as the background image.

- `adminNameLabel`: A label for displaying "Admin Name".
- `adminPasswordLabel`: A label for displaying "Password".
- `adminbglbl`: A label for displaying the background image.
- `errorlblpass`: A label for showing error messages related to the password field.
- `errorlbluser`: A label for showing error messages related to the admin name field.
- `ImagePath`: A string containing the file path for the background image.
- `loginButton`: A button for logging in as admin.
- `updateStatusButton`: A button for updating the status of orders.
- `cancelButton`: A button for canceling expired orders (currently removed in the code).
- `loginPanel`: A panel holding the login-related components.
- `orderPanel`: A panel holding the order-related components.
- `ordersTable`: A JTable for displaying orders in the admin panel.
- `tableModel`: The table model that stores the data for the orders table.
- `foodTable`: A JTable for displaying items in the stock panel.
- `con`: A Connection object for managing the connection to the database.
- `pst`: A PreparedStatement object used for executing SQL queries.

Methods:

- **`YUM_e_AdminModule()`**: Constructor to set up the login UI and initialize components.
- **`actionPerformed(ActionEvent e)`**: Handles actions for the login, order status update, and (currently removed) cancel expired orders.
- **`showOrders()`**: Switches from the login panel to the order management panel and displays the orders.
- **`loadOrders()`**: Loads orders from the database into the orders table.
- **`updateOrderStatus()`**: Updates the status of a selected order in the database.
- **`connect()`**: Establishes a connection to the MySQL database.

- **closeResources()**: Closes the database resources like PreparedStatement and Connection.

Class 2: YUM_e_LogInModule

Purpose:

The YUM_e_LogInModule class handles the user login process for the "YUM-e" food ordering system. It creates a GUI that prompts users for their name and student ID, validates their input, and connects to the database to verify their credentials.

Attributes:

- JTextField usernameField: A text field where users input their name.
- JTextField passwordField: A text field where users input their student ID.
- JLabel usernameLabel, passwordLabel, backgroundimg, errorlblpass, errorlbluser: Labels used for displaying text and error messages.
- JPanel loginPanel: The panel that contains all the components (labels, text fields, button).
- JButton loginButton: The button users click to submit their login details.
- Image bg: Stores the background image for the login screen.
- String imgbg: The file name of the background image.
- Connection con: A connection object for interacting with the database.
- PreparedStatement pst: A prepared statement used to execute SQL queries.

Methods:

- **main(String[] args)**: The entry point of the program, creates and displays the login module frame.
- **Constructor YUM_e_LogInModule()**: Sets up the login screen, adds components like text fields, labels, buttons, and background, and defines actions for the login button.
- **connect()**: Establishes a connection to the database.
- **closeResources()**: Closes the database resources (connection and prepared statement) after use.

Class 3: YUM_e_Menu

Purpose:

The YUM_e_Menu class represents the main menu screen of the YUM_e system. It displays the menu, handles user interactions with the menu items, manages the order, and processes the checkout.

Attributes:

- menubg: The background image for the menu panel.
- menuPanel: The main panel containing all the menu components.
- itemPanel: A panel to display the menu items.
- receiptPanel : A panel to display the order receipt.
- dishScrollPane: A scroll pane to allow scrolling through the menu items.
- receiptScrollPane : A scroll pane to allow scrolling through the receipt.
- drinkImages: An array of image paths for drinks.
- dessertImages : An array of image paths for desserts.
- dishImages: An array of image paths for dishes.
- selectedItems: A map to store the selected menu items and their prices.
- menubglbl: A label to display the background image.
- dishlbl: A label representing the "Dish" category.
- drinkslbl: A label representing the "Drink" category.
- dessertlbl : A label representing the "Dessert" category.
- categorylbl : A label to display the currently selected menu category.
- studentNamelbl : A label to display the student's name.
- studentIDlbl : A label to display the student's ID.
- totalllbl: A label to display the total bill amount.
- orderIDlbl: A label to display the order ID.
- receiptTextArea : A text area to display the order receipt.
- dineInRadio): A radio button for "Dine In" order type.
- takeOutRadio: A radio button for "Take Out" order type.
- orderTypeGroup : A button group to manage the order type radio buttons.
- clear : A button to clear the order.

- **confirm** : A button to confirm the order.
- **add** : A button to add an item to the order.
- **minus** : A button to remove an item from the order.
- **currentOrderType**: Stores the currently selected order type.
- **totalBill** : Stores the total bill amount.
- **studentName** : Stores the student's name.
- **studentID** : Stores the student's ID.
- **orderId**: Stores the current order ID.
- **itemQuantities**: A map to store the quantities of each item in the order.

Methods:

- **main(String[] args)**: The main method creates and displays the menu window.
- **YUM_e_Menu(String studentName, String studentID)**: The constructor initializes the GUI components, sets up the layout, and adds event listeners.
- **createCategoryLabel(String imagePath, String tooltip, int x, int y)**: A helper method to create category labels with hover effects.
- **displayItems(String[] images, String[] names, double[] prices)**: Displays the menu items for a selected category.
- **addItemToReceipt(String itemName, double price)**: Adds an item to the order receipt and updates the total bill.
- **removeItemFromReceipt(String itemName, double price)**: Removes an item from the order receipt and updates the total bill.
- **updateTotalAmountLabel()**: Updates the total bill label with the latest amount.
- **updateReceiptWithOrderType(String orderType)**: Updates the receipt with the selected order type.
- **clearReceipt()**: Clears the order receipt and resets the total bill.
- **checkout()**: Processes the checkout process, validates the order, displays the final receipt, and updates the database.
- **getLastOrderID()**: Retrieves the last order ID from the database.

- **insertOrderIntoDatabase(String studentID, String orderItems, String orderType, double totalBill):** Inserts the order details into the database.
- **calculateTotalBill():** Calculates the total bill amount.
- **getStockLevelFromDatabase(String itemName):** Retrieves the stock level of an item from the database.
- **updateStockLevelInDatabase(String itemName, int quantity):** Updates the stock level of an item in the database after an order.

Key OOP Principles Used

Encapsulation:

Encapsulation in the code is achieved by organizing related data and behaviors into distinct classes, hiding internal details, and exposing only necessary functionalities. For example, the YUM_e_Menu class manages order operations through methods like addItemToReceipt, checkout, and clearReceipt, while keeping the internal data structures (selectedItems, itemQuantities) and their implementation details hidden. The code primarily uses encapsulation by defining private fields in each class, limiting access to data members from external sources. This approach ensures data integrity and enables controlled updates through methods.

Polymorphism

Polymorphism is used in the design through method overriding. The actionPerformed() method is overridden in the YUM_e_AdminModule class. This is a basic form of polymorphism, where different classes provide their implementations of the same method. This occurs when a subclass provides its implementation of a method that is already defined in its parent class. This allows for tailored behavior based on the specific type of object. The actionPerformed() method in the YUM_e_AdminModule class is overridden. This means that the YUM_e_AdminModule class provides its unique implementation of how to respond to button clicks. When a button is clicked in the YUM_e_AdminModule window, it's the overridden actionPerformed() method within this class that gets executed. This contrasts with the actionPerformed() method in other classes, which might handle button clicks in completely different ways.

Inheritance

The code uses inheritance to utilize the features of the Java Swing library, simplifying GUI development. JFrame serves as the base class for components like YUM_e_AdminModule, YUM_e_Menu, and YUM_e_LogInModule, enabling them to create graphical windows. Additionally, GUI components such as JLabel, JButton, and JTextField inherit from Swing's base classes, providing essential functionality for easy implementation. This inheritance is implicit, meaning developers don't need to explicitly define a parent class, enhancing development efficiency. Inheritance offers benefits like reusability, reducing the need to write new code; better code organization through a class hierarchy; and extensibility, allowing developers to add functionality by creating subclasses. In summary, the code effectively utilizes inheritance to make building a user-friendly GUI more efficient with reusable components.

Key Features

The YUM e_Menu Food Ordering System is a user-friendly app that simplifies the food ordering process for students. Its main features include a secure login, a menu display with categorized food items (drinks, desserts, and dishes), and order management. Users can select items, adjust quantities, choose between dine-in or take-out, and see the total price of their order in real-time. The system also processes orders by generating receipts and updating inventory levels to reflect stock changes. Admins have additional features for updating the status and managing the menu and inventory.

The project is built using object-oriented programming (OOP) concepts, which makes it organized and easy to maintain. Encapsulation protects important data, like user credentials and orders, by keeping it private and accessible only through specific methods. Inheritance is applied to create a consistent and reusable user interface by extending GUI classes like JFrame. Polymorphism allows flexibility, such as updating item quantities and recalculating totals based on user actions. These OOP principles ensure the system is simple to use, easy to update, and reliable in managing orders efficiently.

Testing and Results

Testing Methodology

Testing Strategy

Unit Testing

Unit testing for the login, menu, and admin modules was conducted to validate the functionality of individual components and ensure that they operated as expected in isolation. For the login module, tests focused on validating input fields, ensuring that only valid usernames and student IDs were accepted, and verifying error messages for invalid inputs. The menu module's unit tests covered category selection, accurate display of menu items, and proper calculation and generation of receipts. Each button, such as "Confirm" and "Clear," was tested to confirm it triggered the intended actions.

For the admin system, unit tests verified the functionality of features like viewing, updating, and deleting orders. Each action was tested to ensure data integrity and accurate updates to the database. The unit testing process also included validating database connectivity and ensuring that all queries executed correctly. By testing these components independently, potential issues were identified and resolved, ensuring that the modules were robust and ready for integration.

Integration Testing

Integration testing for the YUM_e_LogInModule was conducted to ensure seamless interaction between its components and the database. The testing verified that the login interface accurately captured user input, such as the student name and ID, through the usernameField and passwordField components. These values were then passed to the database query using a prepared statement to validate the student's credentials. It was confirmed that, upon a successful match in the database, the YUM_e_Menu interface launched correctly, displaying the relevant student data. Additionally, the system was tested to ensure it displayed an error message when invalid inputs, such as a non-numeric Student ID or incorrect credentials, were provided.

Database connectivity was another critical aspect that was validated during testing. The connect method successfully established a connection with the MySQL database, while the closeResources method was tested to ensure that database resources were released properly after each login attempt. The integration of UI components, such as the login button and its click event, was also verified to confirm that the background image, labels, and buttons were rendered correctly and performed their designated functions. Overall, the testing confirmed that the login module reliably authenticated users and transitioned them to the menu system when valid credentials were entered.

For the YUM_e_Menu class, it outlined a substantial portion of the GUI for the menu system, offering features for students to place orders, select categories (dishes, drinks, desserts), choose an order type (Dine In/Take Out), and manage receipt generation. This design demonstrated careful thought in creating an intuitive interface for users. During integration testing for the system, several critical points were addressed.

First, the flow from the login window to the admin or student menu window was validated. After a successful login, the system ensured that the correct interface opened: the admin dashboard for the cashier and the menu system (YUM_e_Menu) for students. For students, it was confirmed that their specific data, such as student name and ID, were accurately passed and displayed in the menu interface, maintaining consistency between login credentials and the menu data. In terms of order processing, integration testing verified that the selected menu items and their prices were correctly reflected in the receipt. Additionally, the chosen order type (Dine In or Take Out) dynamically updated the receipt. For database connectivity, the getLastOrderId method was tested to reliably fetch the most recent order ID from the database, and all confirmed order details—including selected items, quantities, and total amounts—were stored accurately in the database.

Category selection was another vital area, requiring the validation of click events for category labels (dishlbl, drinkslbl, dessertlbl) to ensure that the correct items were displayed under each category. Lastly, receipt management functionality underwent thorough testing. Buttons such as "Clear" were tested to ensure they reset the receipt and selected items, while

"Confirm" was validated to save the order details to the database and optionally reset the interface for a new order. These steps ensured a seamless and reliable experience for users.

Moreover, the integration testing for the YUM_e_AdminModule focused on validating the interaction between various components of the application to ensure seamless functionality. The login panel was tested to confirm that it correctly accepted user inputs for the admin name and password and validated them against the database. Error labels for empty input fields and incorrect credentials were evaluated to verify that they appeared appropriately under the intended conditions. The transition from the login panel to the order management panel was tested by successfully authenticating an admin user. The showOrders method was validated to ensure it correctly replaced the login panel with the order management interface and loaded order data from the database into a table. The system was also tested to verify that updating the status of an order, through the updateOrderStatus method, correctly reflected changes both in the table and in the database.

Additionally, the functionality of the "Update Stocks" button was assessed to ensure it opened a new panel displaying food items and stock information retrieved from the database. The integration of the table's data with the database query, along with the ability to interact with individual rows in the table, was validated. Throughout the tests, error handling mechanisms, such as those for database connection failures and invalid input scenarios, were assessed to ensure they provided appropriate feedback to the user.

System Testing

System testing for the login, menu, and admin modules was conducted to ensure end-to-end functionality and a seamless user experience. The login module was tested to verify that it correctly authenticated users by validating their credentials against the database. Successful logins redirected students to the menu module and administrators to the admin module, ensuring that the appropriate interface was displayed based on the user type. The menu module was tested for functionality such as selecting menu categories, placing orders, and generating receipts. It was confirmed that selected items, quantities, and order details were correctly displayed and stored in the database.

The admin system underwent testing to validate its ability to manage student orders, including viewing, updating order statuses, and deleting records. The integration between modules was thoroughly assessed, ensuring smooth transitions between the login, menu, and admin functionalities. Error handling mechanisms, such as displaying appropriate error messages for incorrect login details or database connection failures, were also tested. Overall, the system testing confirmed that all modules worked as intended, with accurate data handling and reliable functionality.

Performance Testing

Performance testing for the login, menu, and admin modules was carried out to evaluate the system's responsiveness, stability, and resource efficiency under various conditions. The login module was tested to ensure it could handle multiple simultaneous login attempts without delays or errors, maintaining a quick response time even during peak usage. The menu module was assessed for its ability to process multiple order placements and category selections efficiently, ensuring that receipt generation and database updates occurred without significant lag.

The admin system was tested for its capacity to manage a high volume of order records, including updating the stocks and order status of students, without performance degradation. Database queries were monitored to confirm they executed efficiently, minimizing delays in data retrieval and storage. Stress testing was conducted to determine the system's behavior under heavy loads, and it was confirmed that the modules maintained consistent performance without crashing or exhibiting significant slowdowns. Overall, the performance testing validated that the system could handle expected workloads effectively.

Test Cases

After unit testing, the modules were combined to test how they interact.

a. Student Login and Menu Integration

Test Scenario: A student logs in with valid credentials. The system should:

1. Authenticate the student by querying the verified_student table in the database.
 2. If student credentials match the data in the database, it will open the menu window and display the items.
- Input:
Name: Emman Abas
Student ID: 546488
 - Expected Outcome: It will open the menu window if the credentials are correct. Otherwise, it will inform the student to enter the correct information.

b. Admin Dashboard and Menu Integration

- Test Scenario: Admin views and manages orders on the dashboard. The system should:
 - Query the database to retrieve all pending and completed orders.
 - Update the status of an order when the admin marks it as "ready."
- Example Input:
 - Action: Click "Update Status" for Order ID 01
 - Action: Mark Order ID 01 as "ready."
- Expected Outcome:
 - The admin sees the order details of the students
 - The database updates the status column for Order ID 01 to "ready."
 - The admin dashboard reflects the updated status.

c. Order Placement and Database Integration

- Test Scenario: A student places an order. The system should:
 - Save the order details, including student ID with its order ID, items, quantities, order_date, order type, and total in the items table.
 - Generate a unique Order ID and associate it with the student's ID.
- Input:
 - Items: Chicken Hotdog (Qty: 1), Fried Chicken (Qty: 1)
 - Total: Php 50.00
- Expected Outcome:

- A new entry appears in the item's table with a unique Order ID.
- The stock table should update the quantity of chicken hotdog and fried chicken in the itemName column.

d. Admin Actions and Database Sync

- Test Scenario: Admin enter the quantity of an item. The system should:
 - Change the item's quantity in the stock table in the database.
 - Update the item's quantity in the food stock table in the admin system.
- Action: Click "Update Stocks"
- Expected Outcome:
 - A new window will pop up, revealing the food stock table
 - Admin can update the item's quantity, changing the quantity in the item's table in the database.

Table 1. YUM_e_LogInModule Test Cases

Test Description	Input/Action taken	Expected Outcome	Actual Outcome	Status
Valid Login	Name: "Emman Abas" Student ID: 546488	Redirect to the YUM_e menu window	Redirect to the YUM_e menu window	PASSED
Invalid Name or Student ID	Name: wrongName Student ID: 231321	Displays the error. "Incorrect input. Please check your name and Student ID."	Displays the error. "Incorrect input. Please check your name and Student ID."	PASSED
Empty Fields	Name: "" Student ID: ""	Displays the error, "Input your valid name and student ID"	Displays the error, "Please enter a valid Student ID and name"	PASSED
Valid Login Without Starting the Apache and mySQL	Name: "Emman Abas" Student ID: 546488	Displays error, "Connection failed. Please check your database settings"	Displays error, "Connection failed. Please check your database settings"	PASSED

Table 2. YUM_e_Menu Test Cases

Test Description	Input/Action taken	Expected Outcome	Actual Outcome	Status
Checked if the menuPanel loaded	Launched the application	Displayed the menu panel with a background color and components	Menu panel loaded successfully with the specified components	PASSED
Checked the studentNameLbl	Passed "Emman Abas" as student name during initialization	Displayed "Emman Abas" at the specified location in the menu panel	Displayed "Emman Abas" correctly	PASSED
Checked the studentIDLbl	Passed "546488" as studentID during initialization	Displayed "546488" at the specified location in the menu panel	Displayed "546488" correctly	PASSED
Interacted with the dishLbl	Clicked on the dishLbl	Displayed a list of dish items along with their images and prices	Displayed the correct items, images, and prices	PASSED
Selected Dine In radio button	Clicked on the Dine In radio button	Updated the receipt with "Order Type: Dine In" and highlighted the button	Receipt updated with "Order Type: Dine In"; button highlight worked as expected	PASSED
Selected Take Out radio button	Clicked on the Take Out radio button	Updated the receipt with "Order Type: Take Out" and highlighted the button	Receipt updated with "Order Type: Take Out"; button highlight worked as expected	PASSED
Cleared the receipt	Clicked on the Clear button	Cleared all items from the receipt and reset total	Receipt cleared and total amount reset	PASSED

		amount	successfully	
Confirmed the order	Added items to the receipt and clicked on Confirm Order	Saved the order details (including items, order type, and total bill) into the database	Order details saved successfully into the database	PASSED
Tested invalid category click	Clicked on an area outside the category labels	Did not change the category label text	Category label text remained unchanged	PASSED
Checked database connectivity	Launched the application with proper database configuration	Successfully connected to the database	Database connected successfully	PASSED
Checked missing database connection	Launched the application without setting up the database	Displayed an error message or exception	Displayed an appropriate error message	PASSED
Checked for duplicate orderID	Placed multiple orders sequentially	Generated unique Order ID for each new order	Unique Order ID generated successfully for all orders	PASSED
Validated receipt update for drinks	Clicked on drinkslbl and selected "Coke Mismo"	Added "Coke Mismo" to the receipt with the correct price	"Coke Mismo" added successfully to the receipt with the price of 20.00	PASSED
Validated receipt update for dessert	Clicked on dessertlbl and selected "Chocolate Cake"	Added "Chocolate Cake" to the receipt with the correct price	"Chocolate Cake" added successfully to the receipt with the price of 20.00	PASSED
Tested receipt scrollability	Added multiple items to the receipt	Displayed all items in the receipt area with scroll functionality	Scroll functionality worked correctly	PASSED

Verified total bill calculation	Selected "Chicken Hotdog" (15.00) and "Coke Mismo" (20.00)	Calculated the total bill as 35.00	Total bill calculated as 35.00 correctly	PASSED
Tested invalid image path	Changed an image path (e.g., /drink1.png to /invalidpath.png) and launched the application	Displayed a placeholder or no image for the invalid path	Placeholder displayed correctly for invalid image paths	PASSED
Checked order ID increment	Placed orders successively	Incremented Order ID correctly after each order	Order ID incremented as expected	PASSED
Tested hover effect on drinkslbl	Hovered the mouse over the drinkslbl	Changed the categorylbl text to "DRINK" with the correct styling	Hover effect updated the label text and styling correctly	PASSED

Table 3. YUM_e_AdminModule Test Cases

Test Description	Input/Action taken	Expected Outcome	Actual Outcome	Status
Admin login with valid credentials	Admin Name: admin, Password: admin123	Displayed orders panel with existing orders.	Displayed orders panel with existing orders.	PASSED
Admin login with invalid credentials	Admin Name: admin, Password: wrongpass	Displayed error message: "Invalid admin credentials."	Displayed error message: "Invalid admin credentials."	PASSED
Admin login with empty fields	Admin Name: (empty), Password: (empty)	Displayed an error message: "Please enter both admin name and password."	Displayed an error message: "Please enter both admin name and password."	PASSED

Updating an order's status	Selected order ID: 101, New status: ready	Updated the selected order's status in the database and in the table.	Updated the selected order's status in the database and in the table.	PASSED
Attempting to update status without selecting an order	No order selected	Displayed error message: "Please select an order to update."	Displayed error message: "Please select an order to update."	PASSED
Displaying food data in the new panel	Clicked "Update Stocks" button	Displayed a panel with food table containing Food ID, Food Name, Price, and Stock.	Displayed a panel with food table containing Food ID, Food Name, Price, and Stock.	PASSED
Handling database connection failure	Simulated database connection error	Displayed error message: "Database error: [specific error message]."	Displayed error message: "Database error: [specific error message]."	PASSED
Loading orders with empty table in the database	No records in items table	Displayed empty orders table with column headers.	Displayed empty orders table with column headers.	PASSED
Displaying error for invalid status update	Selected order ID: 102, Entered status: invalidStatus (not in preparing, ready)	Displayed error message: "Invalid status entered."	Displayed error message: "Invalid status entered."	PASSED
Ensuring valid image path	Valid image path provided for ImagePath	Successfully loaded the login background image	Successfully loaded the login background image	PASSED

Results

The testing phase validated the system's functionality and confirmed alignment with its objectives. The login module reliably authenticates users, providing appropriate feedback for

incorrect inputs and ensuring a seamless transition to the menu or admin dashboard. The menu module accurately processes orders, dynamically calculates totals, and updates receipts based on user selections, while integration testing confirms proper database interactions for data validation and order storage. The admin module effectively supports order and stock management, with features like status updates and inventory modifications functioning as intended. Observations during testing highlighted the system's reliability in handling errors, its intuitive GUI design, and its prompt execution of database operations. However, minor usability issues, such as scrollbar sensitivity and redundant item listings in receipts, were noted. Overall, the results demonstrate that the system is well-implemented, successfully reducing wait times and improving cafeteria service efficiency.

Conclusion

The **YUM-e Food Ordering System** successfully achieved its goal of improving the efficiency of university cafeteria operations by reducing waiting times and enhancing the user experience for students and staff. The system enables students to pre-order meals, pay online, and choose pick-up times while providing administrators with tools to manage orders and inventory seamlessly. The visually appealing and user-friendly interface further ensures ease of navigation for all users.

Throughout the project, several challenges were encountered. Ensuring real-time updates of stock levels required refining the system's database logic, and resolving code errors during module integration and database operations demanded rigorous debugging and collaborative brainstorming. These challenges were addressed through teamwork and the application of learned skills, ultimately enhancing the functionality and reliability of the system.

The project provided a valuable opportunity to apply and deepen our understanding of Object-Oriented Programming (OOP) concepts, including encapsulation, inheritance, and polymorphism, as well as GUI design principles. We gained practical experience in integrating database management, designing intuitive interfaces, and debugging complex systems. This

hands-on approach not only reinforced theoretical knowledge but also honed our problem-solving abilities and teamwork skills.

In conclusion, the project successfully bridges theory and practice, addressing real-world inefficiencies while fostering learning and skill development. Despite the challenges, the team's collective efforts resulted in a functional and robust system, setting the stage for future enhancements and continued growth in software development expertise.

Future Enhancements

For future enhancements or potential expansions of the project, several features could be incorporated to improve the user experience and functionality. First, implementing a flexible payment system would allow students to choose between traditional cash payments or modern online payment methods, providing greater convenience. To further enhance usability, we could add detailed descriptions for each menu item, helping students make informed decisions when placing their orders.

In addition, a feature to view order history could be introduced, enabling users to easily access past orders, complete with details such as items purchased and timestamps. This would help users track their ordering habits and simplify repeat orders.

Another valuable improvement would be to enable students to include special requests in their orders, such as preferences like "no onions" or "extra sauce," offering a more personalized experience.

The integration of a secure and seamless online payment gateway would facilitate cashless transactions, making it more convenient for students to pay directly through the system. Along with online payment, offering a variety of payment methods—such as credit cards, debit cards, and mobile wallets—would provide flexibility and cater to different user preferences, ensuring that the system can accommodate a wider range of payment options.

Furthermore, based on feedback from the survey, several adjustments should be made. The horizontal scrollbar needs to be fixed, and the scrollbar sensitivity should be adjusted for a smoother user experience. The system should also quantify the number of similar food items ordered to avoid redundancy on the receipt, making it clearer and more concise for users. Finally, to improve the order management process, the "clear" button should reset the order increment, ensuring that the system starts fresh with each new order.

Source Code

```
//YUM_e_LogInModule
package YUM.src.project;

import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;

public class YUM_e_LogInModule extends
JFrame implements ActionListener {
    private static JTextField
usernameField, passwordField;
    private static JLabel usernameLabel,
passwordLabel, backgroundimg,
errorlblpass, errorlbluser;
    private static JPanel loginPanel;
    private static JButton loginButton;
    private static Image bg;
    private static String imgbg = "login
background.png";

    static Connection con;
    static PreparedStatement pst;
    public static void main(String[]
args) {
        new
YUM_e_LogInModule().setVisible(true);
    }
    YUM_e_LogInModule() {
        this.setSize(700, 450);
        this.setTitle("Login Module");

        this.setDefaultCloseOperation(EXIT_ON_CL
OSE);
        this.setLocationRelativeTo(null);

        loginPanel = new JPanel();
        loginPanel.setBackground(new
Color(92, 4, 4));

        this.getContentPane().add(loginPanel);
        loginPanel.setLayout(null);

        bg = new
ImageIcon(YUM_e_LogInModule.class.getR
esource(imgbg)).getImage().getScaledInst
ance(700, 415, Image.SCALE_SMOOTH);

        // User Label and Textfield
        usernameLabel = new
JLabel("Name");
        usernameLabel.setBounds(482, 225,
70, 25);
        usernameLabel.setFont(new
Font("Bierstadt", Font.BOLD, 14));
        usernameLabel.setForeground(new
Color(243, 243, 243));
        loginPanel.add(usernameLabel);

        errorlbluser = new JLabel("Please fill
out this field.");

        errorlbluser.setForeground(Color.black); //
White text

        errorlbluser.setHorizontalAlignment(SwingC
onstants.CENTER);
        errorlbluser.setFont(new Font("Arial",
Font.PLAIN, 10));

        errorlblpass.setBounds(477, 250,
140, 25); // Positioned just above the
username field
        errorlblpass.setVisible(false); // Hide
initially
        loginPanel.add(errorlblpass);

        usernameField = new JTextField(50){
            @Override
            protected void
paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D)
g.create();

                g2.setRenderingHint(RenderingHints.KEY_A
NTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                setOpaque(false);
                g2.setColor(getBackground());
                g2.fillRoundRect(0, 0,
getWidth(), getHeight(), 20, 30);
                super.paintComponent(g2);
                g2.dispose();
            }
        };
        usernameField.setBounds(477, 250,
140, 25);

        usernameField.setBorder(BorderFactory.cre
ateCompoundBorder(
            new EmptyBorder(0, 0, 0,
0), new EmptyBorder(0, 5, 0, 0)
        ));
        loginPanel.add(usernameField);

        // Password Label and Textfield
        passwordLabel = new
JLabel("Student ID");
        passwordLabel.setBounds(482, 275,
90, 25);
        passwordLabel.setFont(new
Font("Bierstadt", Font.BOLD, 14));
        passwordLabel.setForeground(new
Color(243, 243, 243));
        loginPanel.add(passwordLabel);

        errorlblpass = new JLabel("Please fill
out this field.");

        errorlblpass.setForeground(Color.black); //
White text

        errorlblpass.setHorizontalAlignment(Swing
Constants.CENTER);
        errorlblpass.setFont(new Font("Arial",
Font.PLAIN, 10));
        errorlblpass.setBounds(477, 300,
140, 25); // Positioned just above the
password field
        errorlblpass.setVisible(false); // Hide
initially
        loginPanel.add(errorlblpass);

        passwordField = new
JPasswordField(50){
            @Override
            protected void
paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D)
g.create();

                g2.setRenderingHint(RenderingHints.KEY_A
NTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                setOpaque(false);
                g2.setColor(getBackground());
                g2.fillRoundRect(0, 0,
getWidth(), getHeight(), 20, 30);
                super.paintComponent(g2);
                g2.dispose();
            }
        };
        passwordField.setBounds(477, 300,
140, 25);

        passwordField.setBorder(BorderFactory.cre
ateCompoundBorder(
            new EmptyBorder(0, 0, 0,
0), new EmptyBorder(0, 5, 0, 0)
        ));
        loginPanel.add(passwordField);

        // Log-in Button
        loginButton = new JButton("Login") {
            @Override
            protected void
paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D)
g.create();

                g2.setRenderingHint(RenderingHints.KEY_A
NTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                setOpaque(false);
                setContentAreaFilled(false);
                g2.setColor(getBackground());
                g2.fillRoundRect(0, 0,
getWidth(), getHeight(), 30, 30);
                super.paintComponent(g2);
                g2.dispose();
            }
        };
        loginButton.setFont(new
Font("Roboto", Font.BOLD, 13));
        loginButton.setBounds(508, 335, 80,
27);
        loginButton.setBackground(new
Color(251, 187, 4));
        loginButton.setBorder(new
EmptyBorder(0, 0, 0, 0));

        loginButton.addActionListener(new
ActionListener() {
            @Override
            public void
actionPerformed(ActionEvent e) {
                String username =
usernameField.getText().trim();
                String studPass =
passwordField.getText().trim();

                if (username.isEmpty()) {
                    // Show error bubble if the
password field is empty
                    errorlbluser.setVisible(true);

                    // Set a timer to hide the
bubble after 3 seconds
                    Timer timer = new
Timer(2000, new ActionListener() {
                        @Override
                        public void
actionPerformed(ActionEvent e) {
                            errorlbluser.setVisible(false); // Hide the
bubble
                        }
                    });
                    timer.setRepeats(false); //
Only execute once
                    timer.start();
                }
            }
        });
    }
}
```



```

        return;
    } else {
        // Hide the error bubble once
        the user fills in the password
        errorlbluser.setVisible(false);
    }
    if (studPass.isEmpty()) {
        // Show error bubble if the
        password field is empty
        errorlblpass.setVisible(true);

        // Set a timer to hide the
        bubble after 3 seconds
        Timer timer = new
        Timer(2000, new ActionListener() {
            @Override
            public void
            actionPerformed(ActionEvent e) {

                errorlblpass.setVisible(false); // Hide the
                bubble
            }
        });
        timer.setRepeats(false); //
        Only execute once
        timer.start();

        return;
    } else {
        // Hide the error bubble once
        the user fills in the password
        errorlblpass.setVisible(false);
    }

    int studID;
    try {
        studID =
        Integer.parseInt(studPass);
    } catch
    (NumberFormatException ex) {

        JOptionPane.showMessageDialog(null,
        "Please enter a valid Student ID.", "Input
        Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    connect();
    try {
        pst =
        con.prepareStatement("SELECT * FROM
        verified_student WHERE student_ID = ?
        AND student_name = ?");
        pst.setInt(1, studID);
        pst.setString(2, username);
        ResultSet rs =
        pst.executeQuery();

        if (rs.next()) {
            YUM_e_Menu yem = new
            YUM_e_Menu(username, studPass);
            yem.setVisible(true);
            dispose();
        } else {

            JOptionPane.showMessageDialog(null,
            "Incorrect input. Please check your name
            and Student ID.", "Login Error",
            JOptionPane.ERROR_MESSAGE);
        }

    } catch (SQLException e2) {

        JOptionPane.showMessageDialog(null,
        "Database error occurred. Please try again
        later.", "Database Error",
        JOptionPane.ERROR_MESSAGE);
        e2.printStackTrace();
    } finally {
        closeResources();
    }
}

    }
});
loginPanel.add(loginButton);

backgroundimg = new JLabel("");
backgroundimg.setIcon(new
ImageIcon(bg));
backgroundimg.setBounds(0, 0, 700,
417);
loginPanel.add(backgroundimg);
} public static void connect() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver")
        ;
        con =
        DriverManager.getConnection("jdbc:mysql:
        //localhost:3306/admin_studentorders",
        "root","");
    } catch (ClassNotFoundException |
        SQLException e) {

        JOptionPane.showMessageDialog(null,"Con
        nection failed. Please check your database
        settings.", "Connection
        Error",JOptionPane.ERROR_MESSAGE);
        e.printStackTrace();
    }
} public static void closeResources() {
    try {
        if (pst != null) pst.close();
        if (con != null) con.close();
    } catch (SQLException e3) {
        e3.printStackTrace();
    }
}

@Override
public void actionPerformed(ActionEvent
e) {

}
}

//YUM_e_AdminModule
package YUM.src.project;

import javax.swing.*.*;
import javax.swing.border.EmptyBorder;
import
javax.swing.table.DefaultTableModel;
import java.awt.*.*;
import java.awt.event.*;
import java.sql.*.*;
import java.util.Timer;
import java.util.TimerTask;

public class YUM_e_AdminModule extends
JFrame implements ActionListener {
    private JTextField adminNameField,
    adminPasswordField;
    private Image adminbg;
    private JLabel adminNameLabel,
    adminPasswordLabel, adminbglbl,
    errorlblpass, errorlbluser;
    private String imagePath =
    "adminbg.png";
    private JButton loginButton,
    updateStatusButton, cancelButton;
    private JPanel loginPanel, orderPanel;
    private JTable ordersTable;
    private DefaultTableModel tableModel;
    static Connection con;
    static PreparedStatement pst;
    static Timer timer;

    public YUM_e_AdminModule() {
        setTitle("Admin Module - Log In");
        setSize(800, 450);
        setLocationRelativeTo(null);
    }
}

setDefaultCloseOperation(JFrame.EXIT_ON
_CLOSE);

loginPanel = new JPanel();
loginPanel.setLayout(null);
loginPanel.setBackground(new
Color(89,0, 4));
add(loginPanel);

adminNameLabel = new
JLabel("Admin Name:");

adminNameLabel.setForeground(Color.WHI
TE);
adminNameLabel.setFont(new
Font("Roboto",Font.BOLD, 12));
adminNameLabel.setBounds(485,
190, 100, 25);
loginPanel.add(adminNameLabel);

adminbg = new
ImageIcon(YUM_e_AdminModule.class.get
Resource(ImagePath)).getImage().getScal
edInstance(790, 470,
Image.SCALE_SMOOTH);

errorlbluser = new JLabel("Please fill
out this field.");

errorlbluser.setForeground(Color.black);

errorlbluser.setHorizontalAlignment(SwingC
onstants.CENTER);
errorlbluser.setFont(new Font("Arial",
Font.PLAIN, 10));
errorlbluser.setBounds(477, 250,
140, 25);
errorlbluser.setVisible(false);
loginPanel.add(errorlbluser);

adminNameField = new
JTextField(20){
    @Override
    protected void
    paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D)
        g.create();

        g2.setRenderingHint(RenderingHints.KEY_A
        NTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
        setOpaque(false);
        g2.setColor(getBackground());
        g2.fillRect(0, 0,
        getWidth(), getHeight(), 20, 30);
        super.paintComponent(g2);
        g2.dispose();
    }
};
adminNameField.setBounds(570,
190, 150, 25);

adminNameField.setBorder(BorderFactory.c
reateCompoundBorder(
    new EmptyBorder(0, 0, 0,
    0), new EmptyBorder(0, 5, 0, 0)
));
loginPanel.add(adminNameField);

adminPasswordLabel = new
JLabel("Password:");
adminPasswordLabel.setBounds(485,
220, 100, 25);

adminPasswordLabel.setForeground(Color.
WHITE);
adminPasswordLabel.setFont(new
Font("Roboto",Font.BOLD, 12));
loginPanel.add(adminPasswordLabel);

```

```

        errorlblpass = new JLabel("Please fill
out this field.");

errorlblpass.setForeground(Color.black);

errorlblpass.setHorizontalAlignment(Swing
Constants.CENTER);
        errorlblpass.setFont(new Font("Arial",
Font.PLAIN, 10));
        errorlblpass.setBounds(477, 300,
140, 25);
        errorlblpass.setVisible(false);
        loginPanel.add(errorlblpass);

        adminPasswordField = new
JPasswordField(20){
    @Override
        protected void
paintComponent(Graphics g) {
            Graphics2D g2 =
(Graphics2D) g.create();

g2.setRenderingHint(RenderingHints.KEY_A
NTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            setOpaque(false);

g2.setColor(getBackground());
            g2.fillRoundRect(0, 0,
getWidth(), getHeight(), 20, 30);
            super.paintComponent(g2);
            g2.dispose();
        }
    };
        adminPasswordField.setBounds(570,
220, 150, 25);

adminPasswordField.setBorder(BorderFacto
ry.createCompoundBorder(
        new EmptyBorder(0, 0, 0,
0), new EmptyBorder(0, 5, 0, 0)
));
        loginPanel.add(adminPasswordField);

        loginButton = new JButton("Login") {
    @Override
        protected void
paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D)
g.create();

g2.setRenderingHint(RenderingHints.KEY_A
NTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            setOpaque(false);
            setContentAreaFilled(false);
            g2.setColor(getBackground());
            g2.fillRoundRect(0, 0, getWidth(),
getHeight(), 25, 30);
            super.paintComponent(g2);
            g2.dispose();
        }
    };
        loginButton.setBounds(600, 250, 90,
25);
        loginButton.setMargin(getInsets());
        loginButton.setBackground(new
Color(251, 187, 4));
        loginButton.addActionListener(this);
        loginButton.setBorder(null);
        loginPanel.add(loginButton);

        adminbglbl = new JLabel();
        adminbglbl.setIcon(new
ImageIcon(adminbg));
        adminbglbl.setBounds(0, 0, 810,
410);

```

```

        loginPanel.add(adminbglbl);

        setVisible(true);
    }

    @Override
        public void actionPerformed(ActionEvent
e) {

            if (e.getSource() ==
loginButton) {
                String adminName =
adminNameField.getText().trim();
                String adminPassword =
adminPasswordField.getText().trim();

                if (adminName.isEmpty() ||
adminPassword.isEmpty()) {

JOptionPane.showMessageDialog(this,
"Please enter both admin name and
password.", "Login Error",
JOptionPane.ERROR_MESSAGE);
                    return;
                }

                try {
                    connect();
                    String query = "SELECT * FROM
admin WHERE admin_name = ? AND
admin_password = ?";

                    pst =
con.prepareStatement(query);
                    pst.setString(1, adminName);
                    pst.setString(2,
adminPassword);
                    ResultSet rs =
pst.executeQuery();

                    if (rs.next()) {
                        showOrders();
                    } else {

JOptionPane.showMessageDialog(this,
"Invalid admin credentials.", "Login Error",
JOptionPane.ERROR_MESSAGE);
                    }

                } catch (SQLException ex) {
                    ex.printStackTrace();

JOptionPane.showMessageDialog(this,
"Database error: " + ex.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
                } finally {
                    closeResources();
                }
            } else if (e.getSource() ==
updateStatusButton) {
                updateOrderStatus();
            } else if (e.getSource() ==
cancelButton) {
                // cancelExpiredOrders(); //REMOVE
            }

            public void showOrders() {
                remove(loginPanel);

                orderPanel = new JPanel();
                orderPanel.setLayout(new
BorderLayout());
                add(orderPanel);

                String[] columnNames = {"Order
ID", "Student ID", "Item", "Status", "Order
Time"};

```

```

                tableModel = new
DefaultTableModel(columnNames, 0);
                ordersTable = new
JTable(tableModel);
                JScrollPane scrollPane = new
JScrollPane(ordersTable);
                orderPanel.add(scrollPane,
BorderLayout.CENTER);

                updateStatusButton = new
JButton("Update Status");

                updateStatusButton.addActionListener(this
);
                orderPanel.add(updateStatusButton,
BorderLayout.SOUTH);

                //REMOVE
                /* cancelButton = new JButton("Cancel
Expired Orders");
                cancelButton.addActionListener(this);
                orderPanel.add(cancelButton,
BorderLayout.NORTH);*/

                loadOrders();
                setVisible(true);
            }

            public void loadOrders() {
                tableModel.setRowCount(0);

                try {
                    connect();
                    String query = "SELECT *
FROM items WHERE status != 'cancelled'";

                    pst =
con.prepareStatement(query);
                    ResultSet rs =
pst.executeQuery();

                    while (rs.next()) {
                        int orderID =
rs.getInt("order_ID");
                        int studentID =
rs.getInt("student_ID");
                        String item =
rs.getString("items");
                        String status =
rs.getString("status");
                        Timestamp orderTime
= rs.getTimestamp("order_date");

                        tableModel.addRow(new Object[]{orderID,
studentID, item, status, orderTime});
                    }

                    // startCancelTimer();
                } catch (SQLException e) {
                    e.printStackTrace();

JOptionPane.showMessageDialog(this,
"Error loading orders: " + e.getMessage(),
"Database Error",
JOptionPane.ERROR_MESSAGE);
                }

                }

            public void updateOrderStatus() {

                int selectedRow =
ordersTable.getSelectedRow();
                if (selectedRow >= 0) {
                    int orderID = (int)
tableModel.getValueAt(selectedRow, 0);
                    String newStatus =
JOptionPane.showInputDialog(this, "Enter
new status (preparing, ready)

```

Personal Information

Name: **Emman Abas**

Date of Birth: **June 25, 2004**

Gender: **Male**

Nationality: **Filipino**

Course and Year: **2nd Year - BS in Computer Science**



Contact Information

Address: **Trading Boulevard, Purok 14, 76-A Bucana Davao City**

Phone Number: **09625828754**

Email Address: **e.abas.546488@umindanao.edu.ph**

Educational Background

- Cesareo Villa-Abrille Elementary School (Grade 1- 6)
- Davao City National High School (Grade 7-8)
- Sta. Ana National High School (Grade 9-10)
- Carlos P. Garcia Senior High School (Grade 11- 12)
- University of Mindanao, Matina Campus (currently 2nd year)

Hobbies and Interests

- Drawing, painting
- Playing video games
- Watching movies
- Listening to music
- Reading books
- Cooking

Personal Information

Name: **Nino Asan**

Date of Birth: **September 11, 2004**

Gender: **Male**

Nationality: **Filipino**

Course and Year: **2nd Year - BS in Computer Science**



Contact Information

Address: **Phase 1 Blk 9 Lot 17, NHA Bangkal, Talomo (Pob.), Davao Del Sur, Davao City, 8000**

Phone Number: **09518937742**

Email Address: **n.asan.546681@umindanao.edu.ph**

Educational Background

- Kimpo Elementary School (Kinder to Grade 6)
- Cotabato City National High School-Main Campus (Grade 7 to 12)
- University of Mindanao, Matina Campus (currently in 2nd year)

Hobbies and Interests

- Playing sports such as volleyball, badminton, chess, lawn tennis, etc.
- Watching movies or series such as KDrama, Harry Potter, Thai series, cartoons, etc.
- I love to explore and travel to different places, taking pictures with them.
- Listening to music of different genres such as KPop, PPop, love songs, etc.

Personal Information

Name: **Joana Bravo**

Date of Birth: **August 02, 2004**

Gender: **Female**

Nationality: **Filipino**

Course and Year: **2nd Year - BS in Computer Science**



Contact Information

Address: **Phase 1, Block 5, Lot 22, San Antonio St., La Verna Hills Subd., Buhangin**

District, Davao City

Phone Number: **09948143839**

Email Address: **j.bravo.546336@umindanao.edu.ph**

Educational Background

- Daycare Learning Center (Kinder)
- San Isidro Elementary School (Grade 2 -3)
- Paradise Embac Elementary School (Grade 1, Grade 4 - 6)
- Arriesgado College Foundation, Inc. (Grade 7)
- Sta. Ana National High School (Grade 8 - 10)
- Carlos P. Garcia Senior High School
- University of Mindanao, Matina Campus (currently 2nd year)

Hobbies and Interests

- Listening to pop and classical music
- Playing puzzle and adventure games

Personal Information

Name: **Hazeljoy T. Hingpit**

Date of Birth: **July 8, 2004**

Gender: **Female**

Nationality: **Filipino**

Course and Year: **2nd Year - BS in Computer Science**



Contact Information

Address: **Phase 2, Block 36 Lot 19, Elenita Heights, Catalunan Grande, Davao City**

Phone Number: **09276409747**

Email Address: **hazeljoyhingpit@gmail.com**

Educational Background

- Heartworks Learning Center (Kinder - Grade 2)
- Davao Macarius Child Academy Inc. (Grade 3 - Grade 6)
- Holy Cross of Mintal Inc. (Grade 7 - 9, Senior High School)
- Catalunan Pequeno National Highschool (Grade 10)
- University of Mindanao, Matina Campus (currently 2nd year)

Hobbies and Interests

- Crochet
- Play Games, Movie,s and Drama
- Korean Pop

Personal Information

Name: **Nazlah Nanding**

Date of Birth: **April 8, 2004**

Gender: **Female**

Nationality: **Filipino**

Course and Year: **2nd Year - BS in Computer Science**



Contact Information

Address: **Deca Homes Mulig 5-27 Davao City**

Phone Number: **09264208310**

Email Address: **nazlahnanding13@gmail.com**

Educational Background

- Usman Baunga Elementary School (Grade 1-3)
- Southville 8 Elementary School (Grade 4-7)
- San Isidro National High School (Grade 7-10)
- Crossing Bayabas National High School (Grade 11-12)
- University of Mindanao, Matina Campus (currently 2nd year)

Hobbies and Interests

- Watching Korean and Chinese series